



PYTHON NOTES

1. History of Python

Python is a high-level programming language created by Guido van Rossum in the late 1980s and officially released in 1991.

Key Points:

- Developed at Centrum Wiskunde & Informatica (CWI) in the Netherlands.
- Named after the TV show “Monty Python’s Flying Circus”, not the snake.
- Designed to be simple, readable, and easy to learn.
- Open-source and free to use.
- Python is maintained by the Python Software Foundation (PSF).

Python Versions:

- Python 1.x – Initial development
- Python 2.x – Widely used but discontinued in 2020
- Python 3.x – Current and recommended version

2. Introduction to Python

Python is a general-purpose, interpreted, high-level language.



Features of Python:

- Easy to read and write
- Uses English-like syntax
- Dynamically typed language
- Portable (runs on Windows, macOS, Linux)
- Supports object-oriented and functional programming

Applications of Python:

- Web Development
- Data Science & Machine Learning
- Artificial Intelligence
- Automation & Scripting
- Game Development
- Cyber Security

3. Setup for Python Using VS Code

Step 1: Install Python

- Download Python from the official website.
- During installation, check “Add Python to PATH”.
- Verify installation: `python --version`

Step 2: Install VS Code

- Download VS Code from its official website.
- Install and open VS Code.



Step 3: Install Python Extension in VS Code

- Open VS Code.
- Go to Extensions (Ctrl + Shift + X).
- Search Python.
- Install the extension by Microsoft.

Step 4: Run Python Program in VS Code

- Create a new file → example.py
- Write Python code.
- Right-click → Run Python File in Terminal

IDEs for Python

- VS Code
- PyCharm
- Jupyter Notebook

Program to Verify Installation

```
import sys
print(sys.version)
```

Python Syntax & Indentation (Expanded)

Rules



- Case-sensitive language
- Statements end with newline
- Indentation = 4 spaces (recommended)

Correct Example

```
if 5 > 3:
```

```
    print("Five is greater")
```

Block Structure Diagram

```
if condition:
```

```
    Statement 1
```

```
    Statement 2
```

4. Variables & Data Types (Expanded)

Variable Characteristics

- Dynamic typing
- Memory allocated at runtime

Example:

```
a = 100
```

```
b = 12.5
```

```
c = "Python"
```

Type Checking

```
print(type(a))
```

```
print(type(b))
```

Mutable vs Immutable

Mutable -> list, dict, set



Immutable -> int, float, tuple, str

5. Operators & Expressions

- Arithmetic Operators
- Relational Opeartors
- Logical Operators

CONDITIONAL STATEMENTS IN PYTHON

What are Conditional Statements?

Conditional statements allow a program to make decisions based on conditions.

They control the flow of execution depending on whether a condition is True or False.

Python uses relational and logical operators to evaluate conditions.

Common Conditions Use:

- Checking marks (pass/fail)
- Validating login
- Comparing numbers
- Decision-based programs

Types of Conditional Statements in Python

- if statement
- if – else statement



- if – elif – else ladder
- Nested if

if Statement

Theory

- Executes a block of code only if the condition is True
- If the condition is False, the block is skipped

Syntax

```
if condition:  
    statements
```

Example Program

```
age = 20  
if age >= 18:  
    print("Eligible to vote")
```

Output

Eligible to vote

Key Points

- Condition must return True or False



- Indentation is compulsory
- No else block

if – else Statement

Theory

- Executes one block if condition is True
- Executes another block if condition is False
- Used when two choices are available

Syntax:

```
if condition:
    true_block
else:
    false_block
```

Example Program

```
num = 10
if num % 2 == 0:
    print("Even Number")
else:
    print("Odd Number")
```

Output

Even Number

Use Cases

- Even or odd



- Pass or fail
- Login success or failure

if – elif – else Ladder (Else-If Ladder)

Theory

- Used when multiple conditions need to be checked
- Conditions are evaluated top to bottom
- First True condition executes, others are skipped

Syntax

```
if condition1:  
    block1elif condition2:  
    block2elif condition3:  
    block3else:  
    default_block
```

Example Program (Grading System)

```
marks = 75  
if marks >= 90:  
    print("Grade A")  
elif marks >= 70:  
    print("Grade B")  
elif marks >= 50:  
    print("Grade C")  
else:  
    print("Fail")
```

Output



Grade B

Important Notes

- else is optional
- Multiple elif allowed
- Improves readability compared to nested if

Nested if Statement

- One if inside another if
- Used when a condition depends on another condition

Syntax

```
if condition1:  
    if condition2:  
        statements
```

Example Program

```
username = "admin"  
password = "1234"  
if username == "admin":  
    if password == "1234":  
        print("Login Successful")  
    else:  
        print("Wrong Password")  
else:  
    print("Invalid Username")
```



LOOPS IN PYTHON

What are Loops?

Loops allow a block of code to execute repeatedly until a condition is met.

Why Loops?

- Avoid repetition
- Reduce code length
- Process collections (list, tuple, string)

Types of Loops in Python

1. for loop
2. while loop
3. Nested loops

for Loop

Theory

- Used to iterate over sequence types
- list, tuple, string, range
- Executes fixed number of times

Syntax

```
for variable in sequence:  
    statements
```



Example Program
for i in range(1, 6):
 print(i)

Output

1
2
3
4
5

Example: Loop through String
for ch in "Python":
 print(ch)

while Loop

Theory

- Executes as long as condition is True
- Condition checked before every iteration
- Used when number of iterations is unknown

Syntax

while condition:



statements

Example Program

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

Output

```
1
2
3
4
5
```

Infinite Loop Example (Be Careful!)

```
while True:
    print("Hello")
```

Loop Control Statements

break

- Terminates the loop immediately

```
for i in range(5):
```

```
    if i == 3:
```

```
        break
```

```
    print(i)
```



continue

- Skips current iteration

```
for i in range(5):  
    if i == 2:  
        continue  
print(i)
```

pass

- Does nothing (placeholder)

```
for i in range(3):  
    pass
```

Nested Loops

Theory

- One loop inside another
- Commonly used for matrices and patterns

Example Program

```
for i in range(3):  
    for j in range(2):  
        print(i, j)
```

Output

```
0 0  
0 1  
1 0
```



1 1
2 0
2 1

Comparison: for vs while

Feature	for loop	while loop
Iterations	Fixed	Unknown
Condition	Implicit	Explicit
Use case	Sequences	Conditions

STRINGS IN PYTHON

What is a String?

A string in Python is a sequence of characters enclosed in:

- Single quotes ' '
- Double quotes " "
- Triple quotes ''' ''' or """ """

Strings are immutable, meaning once created, they cannot be changed.

Example

```
s1 = "Python"
```

```
s2 = 'Programming'
```

```
s3 = """Python is easy"""
```



Characteristics of Strings

- Ordered sequence
- Immutable
- Supports indexing & slicing
- Unicode support
- Iterable (can be used in loops)

Creating Strings

Using Quotes

```
name = "Python"
```

Using str() Constructor

```
num = 123
```

```
s = str(num)
```

```
print(s)
```

Accessing Characters (Indexing)

Positive Indexing

```
s = "Python"print(s[0]) # P
```

```
print(s[3]) # h
```

Negative Indexing

```
print(s[-1]) # n
```

```
print(s[-3]) # h
```



String Slicing

Syntax

```
string[start : end : step]
```

Examples

```
s = "PythonProgramming"  
print(s[0:6])  # Python  
print(s[6:])   # Programming  
print(s[::-1]) # Reverse string
```

String Immutability

Strings cannot be modified after creation.

Invalid Example

```
s = "Python"  
s[0] = 'J'  # Error
```

Correct Way

```
s = "Python"  
s = "J" + s[1:]  
print(s)
```

String Operators

Concatenation (+)

```
a = "Hello"  
b = "World"print(a + b)
```

Repetition (*)



```
print("Hi " * 3)
```

Membership (in, not in)

```
print("Py" in "Python") # True
```

String Functions (Built-in)

- 1. *len()* : Length of string**
- 2. *upper()* : Convert to uppercase**
- 3. *lower()* : Convert to lowercase**
- 4. *title()* : Title case**
- 5. *strip()* : Remove spaces**
- 6. *replace()* : Replace text**
- 7. *split()* : Convert to list**
- 8. *join()* : Join elements**

Example Program

```
s = " python programming "  
print(len(s))  
print(s.upper())  
print(s.strip())  
print(s.replace("python", "java"))
```

String Formatting

```
1. format() Method  
name = "Alice"  
age = 20  
print("Name: {}, Age: {}".format(name, age))
```



2. f-Strings (Recommended)

```
print(f"Name: {name}, Age: {age}")
```

String Comparison

```
a = "apple"
```

```
b = "banana"
```

```
print(a == b)
```

```
print(a < b)
```

Comparison is based on ASCII/Unicode values.

Traversing Strings (Looping)

1. Using for Loop

```
s = "Python"
for ch in s:
    print(ch)
```

2. Using while Loop

```
i = 0
while i < len(s):
    print(s[i])
    i += 1
```

String Methods for Checking

Method	Purpose
--------	---------

isalpha()	—Only letters
-----------	---------------

isdigit()	— Only digits
-----------	---------------

isalnum()	— Letters & digits
-----------	--------------------

islower()	—Lowercase
-----------	------------



isupper()---Uppercase
startswith()---- Start check
endswith()---- End check

Example

```
s = "Python123"print(s.isalnum())  
print(s.startswith("Py"))
```

Counting Characters & Words

Count Characters

```
s = "banana"print(s.count('a'))
```

Count Words

```
sentence = "Python is very easy"  
words = sentence.split()  
print(len(words))
```

Palindrome Program

```
s = input("Enter string: ")  
if s == s[::-1]:  
    print("Palindrome")  
else:  
    print("Not Palindrome")
```

Reverse a String

Using Slicing

```
s = "Python"print(s[::-1])
```

Using Loop

```
rev = ""for ch in s:
```



```
rev = ch + revprint(rev)
```

Common String Programs (Exam Important)

Vowel Count

```
s = "education"  
count = 0  
for ch in s:  
    if ch in "aeiou":  
        count += 1  
print(count)
```

Remove Spaces

```
s = "Hello World"  
print(s.replace(" ", ""))
```

Real-Life Applications of Strings

- User input validation
- Password handling
- Text processing
- File handling
- Web data parsing

LIST IN PYTHON



What is a List?

A list in Python is a collection of ordered elements enclosed in square brackets [].

Lists can store multiple data types and are mutable (can be changed).

Example

```
numbers = [10, 20, 30, 40]
```

```
mixed = [1, "Python", 3.5, True]
```

Characteristics of Lists

- Ordered (maintains insertion order)
- Mutable (elements can be modified)
- Allows duplicate values
- Supports indexing & slicing
- Can store different data types

Creating Lists

Empty List

```
lst = []
```

Using list() Constructor

```
lst = list((1, 2, 3))
```

Nested List

```
matrix = [[1, 2], [3, 4]]
```

List Indexing



Positive Indexing

```
lst = [10, 20, 30, 40]
```

```
print(lst[0]) # 10
```

```
print(lst[2]) # 30
```

Negative Indexing

```
print(lst[-1]) # 40
```

```
print(lst[-3]) # 20
```

List Slicing

Syntax

```
list[start : end : step]
```

Examples

```
lst = [10, 20, 30, 40, 50]
```

```
print(lst[1:4]) # [20, 30, 40]
```

```
print(lst[:3]) # [10, 20, 30]
```

```
print(lst[::2]) # [10, 30, 50]
```

```
print(lst[::-1]) # Reverse list
```

Modifying List Elements (Mutability)

Change Element

```
lst = [10, 20, 30]
```

```
lst[1] = 99
```

```
print(lst)
```

Add Elements

```
lst.append(40)
```

```
lst.insert(1, 15)
```



Remove Elements

`lst.remove(20)`

`lst.pop()`

Common List Methods

<u>Method</u>	<u>Description</u>
<code>append()</code>	Add element at end
<code>insert()</code>	Add at specific index
<code>extend()</code>	Add multiple elements
<code>remove()</code>	Remove specific value
<code>pop()</code>	Remove by index
<code>clear()</code>	Remove all elements
<code>index()</code>	Find position
<code>count()</code>	Count occurrences
<code>sort()</code>	Sort list
<code>reverse()</code>	Reverse list

Example

```
lst = [3, 1, 4, 2]
```

```
lst.sort()
```

```
print(lst)
```

Traversing Lists

Using for Loop

```
lst = [10, 20, 30]
for i in lst:
    print(i)
```

Using while Loop



```
i = 0
while i < len(lst):
    print(lst[i])
    i += 1
```

List Operators

Concatenation

```
a = [1, 2]
b = [3, 4]
print(a + b)
```

Repetition

```
print([1, 2] * 3)
```

Membership

```
print(20 in [10, 20, 30])
```

List Comprehension

Theory

List comprehension provides a short and efficient way to create lists.

Syntax

```
[expression for item in iterable if condition]
```

Examples

```
squares = [x*x for x in range(5)]
even = [x for x in range(10) if x % 2 == 0]
```

Nested Lists

Example

```
matrix = [
    [1, 2, 3],
```




```
[4, 5, 6]
]
for row in matrix:
    for val in row:
        print(val, end=" ")
```

Copying Lists

Shallow Copy

```
a = [1, 2, 3]
b = a.copy()
```

Reference Copy (Be Careful)

```
b = a
```

Sorting Lists

Ascending

```
lst = [5, 2, 9]
lst.sort()
```

Descending

```
lst.sort(reverse=True)
```

Important List Programs (Exam)

Sum of List Elements

```
lst = [10, 20, 30]
print(sum(lst))
```

Find Largest Element



```
lst = [5, 8, 2]
print(max(lst))
```

Remove Duplicates

```
lst = [1, 2, 2, 3]
unique = list(set(lst))
print(unique)
```

Real-Life Applications of Lists

- Storing student marks
- Shopping cart items
- User data storage
- Dynamic data collections

TUPLE IN PYTHON

What is a Tuple?

A tuple in Python is a collection of ordered elements enclosed in parentheses ().

Tuples are immutable, meaning their values cannot be changed after creation.

Example

```
numbers = (10, 20, 30, 40)
```

```
mixed = (1, "Python", 3.5, True)
```



Characteristics of Tuples

- Ordered (maintains insertion order)
- Immutable (cannot be modified)
- Allows duplicate values
- Supports indexing & slicing
- Can store different data types
- Faster than lists (read-only)

Creating Tuples

Empty Tuple

```
t = ()
```

Single Element Tuple

```
t = (10,)
```

Using tuple() Constructor

```
t = tuple([1, 2, 3])
```

Nested Tuple

```
t = ((1, 2), (3, 4))
```

Tuple Indexing

Positive Indexing

```
t = (10, 20, 30, 40)
```

```
print(t[0]) # 10
```

```
print(t[2]) # 30
```

Negative Indexing



```
print(t[-1]) # 40  
print(t[-3]) # 20
```

Tuple Slicing

Syntax

```
tuple[start : end : step]
```

Examples

```
t = (10, 20, 30, 40, 50)
```

```
print(t[1:4])
```

```
print(t[:3])
```

```
print(t[::-1])
```

Tuple Immutability

Tuples cannot be modified

```
t = (10, 20, 30)# t[1] = 99 ❌ Error
```

✓ Convert to list if modification is needed

```
lst = list(t)
```

```
lst[1] = 99
```

```
t = tuple(lst)
```

Common Tuple Methods

Method	Description
count()	Count occurrences
index()	Find position

```
t = (1, 2, 2, 3)  
print(t.count(2))
```



```
print(t.index(3))
```

Traversing Tuples

```
t = (10, 20, 30)
```

```
for i in t:
```

```
    print(i)
```

Tuple Operators

Concatenation

```
print((1, 2) + (3, 4))
```

Repetition

```
print((1, 2) * 3)
```

Membership

```
print(20 in (10, 20, 30))
```

Real-Life Applications of Tuples

- Fixed data (days, months)
- Database records
- Coordinates (x, y)
- Returning multiple values from functions



SET IN PYTHON

What is a Set?

A set is an unordered collection of unique elements, enclosed in curly braces {}.

Example

```
s = {10, 20, 30}
```

Characteristics of Sets

- Unordered
- No duplicate values
- Mutable
- No indexing or slicing
- Faster membership testing

Creating Sets

Empty Set

```
s = set()
```

Using set() Constructor

```
s = set([1, 2, 3])
```

Adding & Removing Elements

Add Element

```
s.add(40)
```



Remove Element

```
s.remove(20)
```

Discard Element (No Error)

```
s.discard(50)
```

Traversing Set

```
for i in s:
```

```
    print(i)
```

Set Operations

Union

```
a = {1, 2}
```

```
b = {2, 3}
```

```
print(a | b)
```

Intersection

```
print(a & b)
```

Difference

```
print(a - b)
```

Symmetric Difference

```
print(a ^ b)
```

Set Membership

```
print(10 in {10, 20, 30})
```



Important Set Programs (Exam)

Remove Duplicates from List

```
lst = [1, 2, 2, 3]
unique = list(set(lst))
print(unique)
```

Real-Life Applications of Sets

- Removing duplicate data
- Mathematical operations
- Finding common items
- User permissions

DICTIONARY IN PYTHON

What is a Dictionary?

A dictionary stores data in key-value pairs, enclosed in { }.

Example

```
student = {"id": 1, "name": "Ram", "marks": 85}
```

```
student = {"id": 1, "id": "Ram", "id": 85} // incorrect
```

Characteristics of Dictionaries

- Stores data as key:value
- Keys are unique



- Mutable
- Ordered (Python 3.7+)
- Fast lookup using keys

Creating Dictionary

Empty Dictionary

```
d = {}
```

Using dict() Constructor

```
d = dict(name="Ram", age=20)
```

Accessing Dictionary Values

```
print(d["name"])
```

Adding & Updating Elements

```
d["marks"] = 90
```

Removing Elements

```
d.pop("age")
```

Traversing Dictionary

```
for k, v in d.items():  
    print(k, v)
```

Common Dictionary Methods

Method	Description
keys()	All keys



values()	All values
items()	Key-value pairs
get()	Safe access
update()	Merge dictionaries
pop()	Remove element
copy()	Copy dictionary

Dictionary Programs (Exam)

Frequency Count

```
lst = [1, 2, 2, 3]
freq = {}
for i in lst:
    freq[i] = freq.get(i, 0) + 1
print(freq)
```

Student Marks Example

```
students = {"Ram": 85, "Shyam": 90}
print(students)
```

Real-Life Applications of Dictionary

- Student records
- Login credentials
- Configuration settings
- Database-like storage

FUNCTIONS IN PYTHON



What is a Function?

A function is a block of reusable code that performs a specific task. Functions help in code reusability, readability, and modularity.

Defining a Function

Syntax

```
def function_name(parameters):  
    statements  
    return value
```

Example

```
def add(a, b):  
    return a + b  
print(add(10, 20))
```

Parameters and Arguments

- Parameters → Variables in function definition
- Arguments → Values passed to function call

```
def greet(name):    # parameter  
    print("Hello", name)
```



```
greet("Python")    # argument
```

Types of Arguments

1. Positional Arguments

```
def sub(a, b):  
    print(a - b)
```

```
sub(10, 5)
```

2. Keyword Arguments

```
sub(b=5, a=10)
```

3. Default Arguments

```
def power(base, exp=2):  
    print(base ** exp)
```

```
power(5)
```

4. Variable Length Arguments

```
def total(*n):  
    print(sum(n))
```

```
total(10, 20, 30)
```

5. Lambda (Anonymous) Functions



Lambda functions are small, one-line functions without a name.

Syntax

lambda arguments : expression

Example

```
square = lambda x: x * x  
print(square(5))
```

6. Scope of Variables

Local Scope

```
def show():  
    x = 10  
    print(x)
```

Global Scope

```
x = 20  
def show():  
    print(x)
```

Global Keyword

```
x = 10  
def change():  
    global x  
    x = 50
```

OBJECT-ORIENTED PROGRAMMING (OOP)

What is OOP?



OOP is a programming approach based on objects and classes. It helps in data security, reusability, and real-world modeling.

Class and Object

Class

A class is a blueprint for creating objects.

Object

An object is an instance of a class.

```
class Student:  
    def display(self):  
        print("Student Object")
```

```
s = Student()  
s.display()
```

Constructor Method (__init__)

The constructor initializes object data.

```
class Student:  
    def __init__(self, name, marks):  
        self.name = name  
        self.marks = marks  
  
    def show(self):  
        print(self.name, self.marks)
```



```
s = Student("Ram", 85)
s.show()
```

Inheritance

Inheritance allows one class to acquire properties of another class.

1. Single Inheritance

One child class inherits from one parent class.

```
class Parent:
    def display(self):
        print("This is Parent Class")
```

```
class Child(Parent):
    def show(self):
        print("This is Child Class")
```

```
obj = Child()
obj.display()
obj.show()
```

Output

```
This is Parent Class
This is Child Class
```



2. Multiple Inheritance

One child class inherits from **more than one parent class**.

```
class Father:
    def skill1(self):
        print("Father skill: Gardening")
```

```
class Mother:
    def skill2(self):
        print("Mother skill: Cooking")
```

```
class Child(Father, Mother):
    def skill3(self):
        print("Child skill: Coding")
```

```
obj = Child()
obj.skill1()
obj.skill2()
obj.skill3()
```

Output

```
Father skill: Gardening
Mother skill: Cooking
Child skill: Coding
```

3. Multilevel Inheritance



Inheritance happens in **multiple levels (Grandparent → Parent → Child)**.

```
class Grandfather:
    def house(self):
        print("Grandfather has a house")
```

```
class Father(Grandfather):
    def car(self):
        print("Father has a car")
```

```
class Child(Father):
    def bike(self):
        print("Child has a bike")
```

```
obj = Child()
obj.house()
obj.car()
obj.bike()
```

Output

```
Grandfather has a house
Father has a car
Child has a bike
```

4. Hierarchical Inheritance



Multiple child classes inherit from the same parent class.

```
class Parent:
    def property(self):
        print("Parent property")

class Child1(Parent):
    def bike(self):
        print("Child1 has bike")

class Child2(Parent):
    def laptop(self):
        print("Child2 has laptop")

obj1 = Child1()
obj2 = Child2()

obj1.property()
obj1.bike()

obj2.property()
obj2.laptop()
```

Output

```
Parent property
Child1 has bike
Parent property
Child2 has laptop
```



5. *Hybrid Inheritance*

Combination of **two or more types of inheritance**.

```
class A:  
    def showA(self):  
        print("Class A")
```

```
class B(A):  
    def showB(self):  
        print("Class B")
```

```
class C(A):  
    def showC(self):  
        print("Class C")
```

```
class D(B, C):  
    def showD(self):  
        print("Class D")
```

```
obj = D()  
obj.showA()  
obj.showB()  
obj.showC()  
obj.showD()
```

Output

```
Class A  
Class B  
Class C
```



Class D

Polymorphism

Polymorphism means many forms.

1. Method Overriding

When a **child class changes the method of the parent class**.

```
class Parent:
    def show(self):
        print("This is Parent class")
```

```
class Child(Parent):
    def show(self):
        print("This is Child class")
```

```
obj = Child()
obj.show()
```

Output

This is Child class

2. Method Overloading (Using Default Arguments)

Python **does not support true method overloading**, but we can achieve it using **default arguments**.

```
class Add:
```



```
def sum(self, a=None, b=None):  
    if a!=None and b!=None:  
        print("Addition:", a+b)  
    elif a!=None:  
        print("Single value:", a)  
    else:  
        print("No values")
```

```
obj = Add()  
obj.sum(10,20)  
obj.sum(5)  
obj.sum()
```

Output

```
Addition: 30  
Single value: 5  
No values
```

3. Operator Overloading

Same **operator** behaves differently depending on data type.

```
print(5 + 3)      # addition  
print("Hello " + "World") # string concatenation
```



```
print([1,2] + [3,4])    # list merge
```

Output

```
8
Hello World
[1, 2, 3, 4]
```

Custom Operator Overloading:

```
class Number:
    def __init__(self, num):
        self.num = num

    def __add__(self, other):
        return self.num + other.num
```

```
n1 = Number(10)
n2 = Number(20)
```

```
print(n1 + n2)
```

Output

```
30
```

4. Function Polymorphism

Same function works with **different data types**.



```
print(len("Python"))  
print(len([1,2,3,4]))  
print(len((10,20,30)))
```

Output

```
6  
4  
3
```

Another example:

```
def multiply(a,b):  
    return a*b
```

```
print(multiply(5,4))  
print(multiply("Hi ",3))
```

Output

```
20  
Hi Hi Hi
```

5. Duck Typing in Python (Polymorphism)

Duck Typing means:

If an object **behaves like a method**, Python will accept it **without checking its type**.



The famous rule is:

"If it walks like a duck and quacks like a duck, it is a duck."

That means Python **does not check the object type**, it only checks **whether the required method exists**.

1. Simple Duck Typing Example

```
class Duck:
```

```
    def sound(self):  
        print("Duck quacks")
```

```
class Dog:
```

```
    def sound(self):  
        print("Dog barks")
```

```
def make_sound(animal):  
    animal.sound()
```

```
d = Duck()  
dg = Dog()
```

```
make_sound(d)  
make_sound(dg)
```

Output

```
Duck quacks  
Dog barks
```

Both objects work because they have the **same method** **sound()**.



2. Duck Typing Example with Different Classes

```
class Student:
    def info(self):
        print("Student information")
```

```
class Teacher:
    def info(self):
        print("Teacher information")
```

```
def display(obj):
    obj.info()
```

```
s = Student()
t = Teacher()
```

```
display(s)
display(t)
```

Output

```
Student information
Teacher information
```

3. Real-Life Example (Payment System)

```
class PhonePe:
    def pay(self):
        print("Payment done using PhonePe")
```



```
class GooglePay:
    def pay(self):
        print("Payment done using GooglePay")
```

```
class Paytm:
    def pay(self):
        print("Payment done using Paytm")
```

```
def payment(app):
    app.pay()
```

```
p1 = PhonePe()
p2 = GooglePay()
p3 = Paytm()
```

```
payment(p1)
payment(p2)
payment(p3)
```

Output

```
Payment done using PhonePe
Payment done using GooglePay
Payment done using Paytm
```

✓ Python only checks **whether pay() method exists.**

Encapsulation



Encapsulation means binding data and methods together and hiding data.

```
class Bank:
```

```
    def __init__(self):  
        self.__balance = 1000  # private
```

```
    def get_balance(self):  
        return self.__balance
```

7. Practical OOP Example

```
class Employee:
```

```
    def __init__(self, name, salary):  
        self.name = name  
        self.salary = salary
```

```
    def display(self):  
        print(self.name, self.salary)
```

```
e = Employee("Amit", 30000)  
e.display()
```

FILE HANDLING IN PYTHON

What is File Handling?



File handling allows Python to read and write data permanently.

File Operations

Operation	Description
<code>open()</code>	Open file
<code>read()</code>	Read data
<code>write()</code>	Write data
<code>close()</code>	Close file

Writing to a File (Text)

```
f = open("data.txt", "w")
f.write("Hello Python")
f.close()
```

Reading from a File

```
f = open("data.txt", "r")
print(f.read())
f.close()
```

File Handling Using with Statement

```
with open("data.txt", "r") as f:
    print(f.read())
```

CSV File Example

```
import csv
with open("data.csv", "w", newline="") as f:
```



```
writer = csv.writer(f)
writer.writerow(["Name", "Marks"])
```

JSON File Example

```
import json

data = {"name": "Ram", "age": 20}
with open("data.json", "w") as f:
    json.dump(data, f)
```

EXCEPTION HANDLING

What is Exception Handling?

Exception handling prevents program termination due to runtime errors.

try-except Block

```
try:
    x = int(input())
    print(10 / x)except:
    print("Error Occurred")
```

try-except-finally



```
try:  
    print(10 / 0)except:  
    print("Error")finally:  
    print("Always Executes")
```

Creating Custom Exceptions

```
class AgeError(Exception):  
    pass
```

```
age = int(input())  
if age < 18:  
    raise AgeError("Not Eligible")
```

MODULES, PACKAGES & ENVIRONMENTS

1. What is a Module?

A module is a file containing Python code.

2. Importing Built-in Modules

math Module

```
import mathprint(math.sqrt(25))
```

datetime Module

```
import datetimeprint(datetime.date.today())
```



os Module
import osprint(os.getcwd())

3. Creating Custom Module

```
mymodule.py
def add(a, b):
    return a + b
main.py
import mymoduleprint(mymodule.add(5, 3))
```

4. Packages

A package is a folder containing modules and `__init__.py`.

```
mypackage/
├── __init__.py
├── mod1.py
└── mod2.py
```

5. Virtual Environment (venv)

A virtual environment is an isolated Python environment.

Create venv

```
python -m venv myenv
```

Activate

```
myenv\Scripts\activate
```